CHAPTER 18

Kinesis and Analytics

Data Streams · Firehose · Flink · Athena · Real-Time Processing · ShopFlow

18-01 Why Real-Time Analytics Matters for ShopFlow

■ Real-Time Analytics with Amazon Kinesis

Traditional batch analytics (nightly reports) tells you what happened yesterday. Real-time analytics tells you what is happening right now. ShopFlow needs to know: which products are trending in the last 5 minutes (so the recommendation engine can surface them), what the checkout conversion rate is this second (so the growth team can react instantly), and whether a fraud pattern is emerging (so orders can be blocked in real-time).

Example: Black Friday: Without real-time analytics, ShopFlow only discovers trending products the next morning. With Kinesis + real-time processing, the product-svc recommendation engine detects that "Laptop Stand Black" is being added to carts 300 times per minute (up from 10/min) and surfaces it at the top of search results — within 30 seconds of the trend starting. Sales of that product increase 40% more than without real-time.

Analytics Type	Latency	ShopFlow Use Case	AWS Service
Real-time streaming	< 1 second	Trending products, fraud detection, live dashboard, checkout conversion	Kinesis Data Streams + Lambda or Managed Fli
Near real-time	1-60 seconds	Business KPI dashboards (revenue/min), inventory alerts	Kinesis Firehose -> S3 -> Athena; or EventBridg
Micro-batch	1-15 minutes	Email triggers (cart abandonment after 5 min), restock alerts	SQS + Lambda (scheduled invocation)
Batch (ETL)	Hours to daily	Historical trend analysis, ML model training, financial reports	Glue ETL -> S3 -> Athena; or Redshift

18-02 Kinesis Data Streams — Core Concepts

Kinesis Data Streams is a real-time data streaming service. Data records are written to the stream by producers and read by consumers. Data is retained for 24 hours (extendable to 7 days or 365 days).

Concept	Definition	ShopFlow Example
Stream	A named, ordered sequence of records with configurable capacity	shopflow-user-events stream: receives all user actions (views, cart a
Shard	The unit of capacity in a stream. 1 shard = 1 MB/s write + 2 MB/s read. More shards = more throughput	shopflow-user-events: 10 shards = 10 MB/s write capacity for 1M MA
Record	One data item: partition key + data blob (up to 1 MB)	{"userId":"USR-123","action":"ADD_TO_CART","productId":"PROD-4
Partition Key	Routes the record to a specific shard. Same key = same shard = ordered delivery for that key	userId as partition key: all events from the same user go to the same shar
Sequence Number	Auto-assigned unique ID per record within a shard. Allows consumers to track their position	Consumer reads from sequence number 495903882714902566085596925
Consumer	Application that reads from the stream. Can be Lambda (event source mapping), Kinesis Data Analytics (Flink), or cus	Lambda function reads shopflow-user-events to update real-time trending
Retention Period	How long records stay in the stream before deletion. Default 24 hours, max 365 days	Set to 7 days: allows reprocessing if Lambda function has a bug

18-03 ShopFlow Event Stream Design

ShopFlow publishes all user interactions as events to Kinesis. Three streams handle different event categories with different throughput and processing requirements.

Stream Name	Events Published	Throughput	Consumers	Retention
shopflow-user-events	Product views, searches, add-to-cart, wishlist adds	High: 1,000 events/sec during sale	Trending engine (Lambda), Personalization (Flink), Firehose	7 days

Stream Name	Events Published	Throughput	Consumers	Retention
shopflow-order-events	Order placed, payment success/fail, order shipped, delivered	Medium: 10-100 events/sec	Fraud detection (Lambda), inventory update (Lambda), anal	7 days
shopflow-clickstream	Browser mouse moves, page views, scroll depth, session events	Very high: 5,000+ events/sec	Session analytics (Flink), A/B test analysis	24 hours (high v

18-04 Kinesis Console — Creating a Data Stream

Follow these steps to create the shopflow-user-events stream in the Kinesis Console.

■ Create Kinesis Data Stream

1. AWS Console -> Kinesis -> Data Streams -> Create data stream
2. Data stream name: shopflow-user-events
3. Capacity mode: Provisioned (predictable load) OR On-demand (auto-scaling)
4. For On-demand: no configuration needed; Kinesis scales automatically
5. For Provisioned: Number of open shards: 10 (1 MB/s write per shard)
6. Click Create data stream
7. Wait ~30 seconds for stream to become Active
8. Click on stream name -> Monitoring tab shows IncomingBytes, PutRecord.Success metrics
9. Click Configuration tab -> note Stream ARN (needed for CDK and IAM policies)

■ aws › Kinesis > Data Streams > shopflow-user-events

Data Stream: shopflow-user-events

■ Stream Details

■ Status: Active | Mode: Provisioned | Shards: 10

■ Stream ARN: arn:aws:kinesis:us-east-1:123456789:stream/shopflow-user-events

■ Retention: 7 days (168 hours)

■ Capacity

■ Write: 10 MB/s (10 shards x 1 MB/s) | 10,000 records/s

■ Read (shared): 20 MB/s | Read (enhanced fan-out): 20 MB/s per consumer

■ Enhanced Fan-out Consumers

■ shopflow-trending-engine-consumer | Registered | 2.0 MB/s dedicated throughput

■ shopflow-personalization-consumer | Registered | 2.0 MB/s dedicated throughput

18-05 Publishing Events from Spring Boot to Kinesis

ShopFlow's Spring Boot microservices publish events to Kinesis using the AWS SDK. The event is serialized to JSON and published as a Kinesis record. This happens asynchronously so it does not slow down the user-facing API response.

```
// Spring Boot: Publish user events to Kinesis asynchronously @Service public class
UserEventPublisher { private final KinesisAsyncClient kinesis = KinesisAsyncClient.create(); //
```

```
Call this method after each user action @Async // runs in a thread pool; does not block the HTTP response thread public void publishEvent(UserEvent event) { try { String json = objectMapper.writeValueAsString(event); kinesis.putRecord(PutRecordRequest.builder() .streamName("shopflow-user-events") .partitionKey(event.getUserId()) // same user -> same shard -> ordered .data(SdkBytes.fromUtf8String(json)) .build()) .thenAccept(response -> log.debug("Event published: seqNo={}", response.sequenceNumber())) .exceptionally(ex -> { log.error("Failed to publish event: {}", event, ex); // Don't throw: event publishing failure should not fail the API response return null; }); } catch (JsonProcessingException e) { log.error("Failed to serialize event", e); } } } // Event record schema (published to Kinesis) public record UserEvent( String userId, String sessionId, String action, // PRODUCT_VIEW, ADD_TO_CART, SEARCH, CHECKOUT_START String productId, // null for SEARCH events String searchQuery, // null for non-SEARCH events String category, Instant timestamp, Map[String, String] metadata // additional context ) {} // Example JSON in Kinesis: // {"userId":"USR-123","action":"ADD_TO_CART","productId":"PROD-456", // "category":"Electronics","timestamp":"2026-06-17T14:23:01Z"}
```

18-06 Lambda as Kinesis Consumer — Trending Products Engine

Lambda functions can be triggered by Kinesis Data Streams with configurable batch sizes and window sizes. ShopFlow's trending engine reads user events and updates a Redis sorted set with product view counts.

```
// Lambda: Trending products engine (Kinesis -> Redis sorted set) @LambdaHandler public class TrendingEngineHandler { private final JedisPool redisPool; private final ObjectMapper mapper = new ObjectMapper(); // Lambda invoked with a batch of Kinesis records (up to 1000 per invocation) public void handleRequest(KinesisEvent event, Context ctx) { Map[String, Long] productViewCounts = new HashMap(); // Count product views in this batch for (KinesisEvent.KinesisEventRecord record : event.getRecords()) { String json = record.getKinesis().getData().toString(); UserEvent userEvent = mapper.readValue(json, UserEvent.class); if ("PRODUCT_VIEW".equals(userEvent.action()) || "ADD_TO_CART".equals(userEvent.action())) { productViewCounts.merge(userEvent.productId(), 1L, Long::sum); } } // Increment counts in Redis sorted set (TTL: sliding 5-minute window) try (Jedis jedis = redisPool.getResource()) { String key = "trending:5min:" + currentFiveMinuteWindow(); for (Map.Entry[String, Long] entry : productViewCounts.entrySet()) { // ZINCRBY: increment score for productId in sorted set jedis.zincrby(key, entry.getValue(), entry.getKey()); } // Expire key after 10 minutes (5-min window + grace period) jedis.expire(key, 600); } } private String currentFiveMinuteWindow() { // Returns "2026-06-17T14:20" (rounds down to 5-minute bucket) long minutes = Instant.now().getEpochSecond() / 60; long bucket = (minutes / 5) * 5; return Instant.ofEpochSecond(bucket * 60).toString().substring(0, 16); } } // Product search API reads trending from Redis: // ZREVRANGE trending:5min:2026-06-17T14:20 0 9 WITHSCORES // Returns top 10 trending product IDs with view count scores
```

18-07 Kinesis Data Firehose — Stream to S3/Redshift

Kinesis Firehose delivers streaming data to S3, Redshift, OpenSearch, or HTTP endpoints. Unlike Kinesis Data Streams (where you manage consumers), Firehose is fully managed — you just configure a destination.

Aspect	Kinesis Data Streams	Kinesis Data Firehose
Consumer management	You write consumer applications (Lambda, Flink, custom)	Fully managed delivery to configured destination — no cod
Latency	Real-time: milliseconds to seconds	Near real-time: 60-900 second buffer before delivery to S3
Data transformation	Custom code in consumer	Optional: Lambda transformation before delivery (e.g., add
Destination options	Any application you write	S3, Redshift, OpenSearch, Splunk, HTTP endpoint, Datado
Retry / error handling	Consumer handles retries	Firehose retries automatically; failed records to error S3 pre
ShopFlow use	Real-time processing (trending engine, fraud detection)	Batch delivery of events to S3 for Athena/Glue analytics

18-08 Firehose CDK — Event Archive to S3

ShopFlow uses Firehose to archive all user events to S3 in Parquet format. Parquet is columnar and 10-20x cheaper to query with Athena than raw JSON.

```
// CDK: Kinesis Firehose -> S3 with Parquet conversion CfnDeliveryStream firehose =
CfnDeliveryStream.Builder.create(this, "UserEventsFirehose")
.deliveryStreamName("shopflow-user-events-archive")
.deliveryStreamType("KinesisStreamAsSource") // Source: read from Kinesis Data Stream
.kinesisStreamSourceConfiguration(
CfnDeliveryStream.KinesisStreamSourceConfigurationProperty.builder()
.kinesisStreamArn(userEventsStream.getStreamArn()) .roleArn(firehoseRole.getRoleArn())
.build()) // Destination: S3 with Parquet conversion (via Glue schema)
.extendedS3DestinationConfiguration(
CfnDeliveryStream.ExtendedS3DestinationConfigurationProperty.builder()
.bucketArn(analyticsDataBucket.getBucketArn())
.prefix("user-events/year=!{timestamp:yyyy}/month=!{timestamp:MM}/day=!{timestamp:dd}/")
.errorOutputPrefix("user-events-errors/!{firehose:error-output-type}/")
.roleArn(firehoseRole.getRoleArn())
.bufferingHints(CfnDeliveryStream.BufferingHintsProperty.builder() .intervalInSeconds(300) //
deliver every 5 minutes .sizeInMBs(128) // OR when buffer reaches 128 MB .build()) // Convert
JSON to Parquet (much cheaper for Athena queries) .dataFormatConversionConfiguration(
CfnDeliveryStream.DataFormatConversionConfigurationProperty.builder()
.inputFormatConfiguration( CfnDeliveryStream.InputFormatConfigurationProperty.builder()
.deserializer(CfnDeliveryStream.DeserializerProperty.builder()
.openXJsonSerDe(Map.of()).build()) .build()) .outputFormatConfiguration(
CfnDeliveryStream.OutputFormatConfigurationProperty.builder()
.serializer(CfnDeliveryStream.SerializerProperty.builder()
.parquetSerDe(CfnDeliveryStream.ParquetSerDeProperty.builder() .compression("SNAPPY").build())
.build()) .build())
.schemaConfiguration(CfnDeliveryStream.SchemaConfigurationProperty.builder()
.databaseName("shopflow_events") .tableName("user_events") // Glue table with schema
.roleArn(firehoseRole.getRoleArn()) .build()) .enabled(true) .build()) .build()) .build();
```

18-09 Amazon Athena — Query S3 Data with SQL

Athena is a serverless SQL query engine for data in S3. ShopFlow uses Athena to analyze archived Kinesis events — answering business questions without loading data into a database.

```
-- Create Athena table over Kinesis Firehose Parquet output CREATE EXTERNAL TABLE
shopflow_events.user_events ( userId STRING, sessionId STRING, action STRING, productId STRING,
searchQuery STRING, category STRING, timestamp TIMESTAMP ) STORED AS PARQUET LOCATION
"s3://shopflow-analytics-data/user-events/" TBLPROPERTIES ("parquet.compression" = "SNAPPY");
-- Partition projection (Athena reads only relevant partitions automatically) ALTER TABLE
shopflow_events.user_events SET TBLPROPERTIES ( "projection.enabled" = "true",
"projection.year.type" = "integer", "projection.year.range" = "2025,2030",
"projection.month.type" = "integer", "projection.month.range" = "1,12", "projection.day.type" =
"integer", "projection.day.range" = "1,31", "storage.location.template" =
"s3://shopflow-analytics-data/user-events/year=${year}/month=${month}/day=${day}/" ); --
Business question: What are the top 10 searched terms today? SELECT searchQuery, COUNT(*) as
searchCount FROM shopflow_events.user_events WHERE action = "SEARCH" AND year = 2026 AND month =
6 AND day = 17 -- partition pruning (only reads today's data) GROUP BY searchQuery ORDER BY
searchCount DESC LIMIT 10; -- Business question: Checkout funnel conversion rate this week
SELECT COUNT(CASE WHEN action = "CHECKOUT_START" THEN 1 END) as checkoutStarts, COUNT(CASE WHEN
action = "ORDER_PLACED" THEN 1 END) as ordersPlaced, ROUND(100.0 * COUNT(CASE WHEN action =
"ORDER_PLACED" THEN 1 END) / COUNT(CASE WHEN action = "CHECKOUT_START" THEN 1 END), 1) as
conversionRate FROM shopflow_events.user_events WHERE year = 2026 AND month = 6;
```

18-10 Amazon Managed Flink — Real-Time Stream Processing

Amazon Managed Service for Apache Flink (formerly Kinesis Data Analytics for Apache Flink) runs Apache Flink jobs on managed infrastructure. ShopFlow uses Flink for complex stream processing: sessionization, joins between streams, and windowed aggregations.

```
// Flink: Real-time checkout abandonment detection // When a user starts checkout but doesn't
complete within 10 minutes -> trigger email DataStream[UserEvent] events = env.addSource( new
FlinkKinesisConsumer[UserEvent]( "shopflow-user-events", new UserEventDeserializationSchema(),
kinesisConsumerConfig)); // Keyed stream: one state per userId KeyedStream[UserEvent, String]
keyedEvents = events .keyBy(UserEvent::getUserId); // Session window: group events within
10-minute idle gap DataStream[CheckoutAbandonment] abandonments = keyedEvents
.window(EventTimeSessionWindows.withGap(Time.minutes(10))) .process(new
```

```
CheckoutAbandonmentDetector()); // CheckoutAbandonmentDetector: Flink ProcessWindowFunction
public class CheckoutAbandonmentDetector extends ProcessWindowFunction[UserEvent,
CheckoutAbandonment, String, TimeWindow] { @Override public void process(String userId, Context
ctx, Iterable[UserEvent] events, Collector[CheckoutAbandonment] out) { boolean startedCheckout
= false; boolean completedOrder = false; UserEvent lastCartEvent = null; for (UserEvent e :
events) { if ("CHECKOUT_START".equals(e.action())) startedCheckout = true; if
("ORDER_PLACED".equals(e.action())) completedOrder = true; if
("ADD_TO_CART".equals(e.action())) lastCartEvent = e; } // Session ended (10-min idle gap) but
checkout started and never completed if (startedCheckout && !completedOrder && lastCartEvent !=
null) { out.collect(new CheckoutAbandonment(userId, lastCartEvent.productId(),
ctx.window().getEnd())); } } } // Output to SNS: triggers "Complete your purchase" email via SES
abandonments.addSink(new SnsSink[CheckoutAbandonment](...));
```

18-11 ShopFlow Real-Time Analytics Architecture

Here is the complete streaming analytics architecture for ShopFlow.

18-12 Kinesis CDK — Complete Stream Configuration

ShopFlow's Kinesis streams and Lambda consumers are defined in CDK for consistent configuration.

```
// CDK: Kinesis Data Streams + Lambda consumer Stream userEventsStream =
Stream.Builder.create(this, "UserEventsStream") .streamName("shopflow-user-events")
.shardCount(10) // 10 MB/s write; adjust based on event volume
.retentionPeriod(Duration.days(7)) .streamMode(StreamMode.PROVISIONED) // On-demand mode:
.streamMode(StreamMode.ON_DEMAND) (auto-scales; no shardCount needed)
.encryption(StreamEncryption.KMS) // encrypt at rest .encryptionKey(Key.Builder.create(this,
"StreamKey").build()) .build(); // Lambda consumer: trending engine Function
trendingEngineLambda = Function.Builder.create(this, "TrendingEngine")
.functionName("shopflow-trending-engine") .runtime(Runtime.JAVA_21)
.handler("com.shopflow.analytics.TrendingEngineHandler::handleRequest")
.code(Code.fromAsset("lambdas/trending-engine/target/trending-engine.jar")) .memorySize(512)
.timeout(Duration.seconds(60)) .environment(Map.of("REDIS_HOST",
redisCluster.getAttrRedisEndpointAddress())) .build(); // Event source mapping: Kinesis ->
Lambda (batch of 100 records, 10s window)
trendingEngineLambda.addEventSource(KinesisEventSource.Builder.create(userEventsStream)
.startingPosition(StartingPosition.LATEST) .batchSize(100) // process up to 100 events per
Lambda invocation .bisectBatchOnFunctionError(true) // on error: retry half the batch to
isolate bad records .retryAttempts(3) .tumblingWindow(Duration.seconds(10)) // accumulate
events for 10s before invoking .build()); // Grant Lambda permission to read from Kinesis
userEventsStream.grantRead(trendingEngineLambda);
```

18-13 Hands-On Lab — Create Stream and Lambda Consumer

In this lab you will create a Kinesis Data Stream, publish test events, and create a Lambda function that reads and processes them. Time estimate: 45 minutes.

■ Lab: Kinesis Stream + Lambda Consumer

1. Kinesis -> Create data stream -> Name: shopflow-lab -> On-demand mode -> Create
2. Test publish via CLI: aws kinesis put-record --stream-name shopflow-lab --partition-key user1 --data "eyJ1c2VySWQiOiJVU1ItMSIsImFjdGlvbiI6IlZJRVcifQ=="
3. (That base64 decodes to: {"userId":"USR-1","action":"VIEW"})
4. Lambda -> Create function -> Author from scratch -> Node.js 20 (or Java 21)
5. Function code: log each Kinesis record to CloudWatch (event.Records.forEach(r => console.log(Buffer.from(r.kinesis.data, "base64").toString())))
6. Lambda -> Add trigger -> Kinesis -> Stream: shopflow-lab -> Batch size: 10 -> Enable
7. Publish 5 test records via CLI: loop with aws kinesis put-record
8. Lambda -> Monitor -> View logs in CloudWatch -> see your JSON records
9. Kinesis Console -> Monitoring tab -> see IncomingRecords metric spike
10. Cleanup: delete Lambda function, Kinesis stream

18-14 Kinesis and Analytics Cost Analysis

Service	Pricing	ShopFlow Monthly Volume	Monthly Cost
Kinesis Data Streams — Shards	$0.015/shard-hour	13 shards x 730 hours	$142
Kinesis Data Streams — PUT payload units	$0.014 per 1M PUT payload units (25KB each)	50M events x avg 0.5 KB = 1M payload units	$14
Kinesis Enhanced Fan-out	$0.015/shard-hour per consumer + $0.013/GB	2 consumers x 13 shards x 730h + 1.5 TB data	$285 + $20
Kinesis Firehose	$0.029/GB ingested + $0.019/GB for format conversion	30 GB/day x 30 days + Parquet conversion	$26 + $17
Lambda (Kinesis consumer)	$0.20/M invocations + $0.0000166667/GB-second	5M invocations/month x 512MB x 0.1s avg	$1 + $4.25

Service	Pricing	ShopFlow Monthly Volume	Monthly Cost
Athena queries	$5/TB scanned (Parquet is 10-20x cheaper than JSON)	100 queries x 1 GB Parquet scanned each (was 20 GB JSON)	$0.50 (vs $10 for JSON)
Managed Flink	$0.11/KPU/hour (1 KPU = 1 vCPU + 4 GB RAM)	2 KPUs for checkout abandonment Flink job	$160
TOTAL	—	—	~$650/month

18-15 Real-Time Fraud Detection with Kinesis + Lambda

ShopFlow uses Kinesis + Lambda to detect fraudulent orders in real-time. Orders with a fraud score > 80 are held for review or automatically declined.

```
// Lambda: Real-time fraud scoring on order events @LambdaHandler public class FraudDetectionHandler { public void handleRequest(KinesisEvent event, Context ctx) { for (KinesisEvent.KinesisEventRecord record : event.getRecords()) { OrderEvent order = parseOrder(record); if (!"ORDER_PLACED".equals(order.action())) continue; int fraudScore = calculateFraudScore(order); if (fraudScore > 80) { // Hold order for manual review dynamoDB.updateItem(UpdateItemRequest.builder() .tableName("shopflow-orders") .key(Map.of("orderId", AttributeValue.fromS(order.orderId()))) .updateExpression("SET orderStatus = :status, fraudScore = :score") .expressionAttributeValues(Map.of( ":status", AttributeValue.fromS("HELD_FOR_REVIEW"), ":score", AttributeValue.fromN(String.valueOf(fraudScore)))) .build()); // Notify fraud review team sns.publish(PublishRequest.builder() .topicArn(FRAUD_ALERT_TOPIC) .subject("Fraud Alert: Order " + order.orderId()) .message(order.toString()) .build()); } } } private int calculateFraudScore(OrderEvent order) { int score = 0; // Rule 1: New account (< 7 days) placing large order if (order.accountAgeDays() < 7 && order.orderValueUsd() > 500) score += 40; // Rule 2: Shipping to different country than billing if (!order.billingCountry().equals(order.shippingCountry())) score += 25; // Rule 3: More than 3 orders in last hour from same IP if (getOrderCountByIpLastHour(order.clientIp()) > 3) score += 35; // Rule 4: High-risk product category (gift cards, electronics) if ("GIFT_CARDS".equals(order.category())) score += 20; return Math.min(score, 100); } }
```

18-16 Amazon QuickSight — Business Intelligence Dashboards

QuickSight is AWS's managed BI (Business Intelligence) service. ShopFlow uses QuickSight to build dashboards for the business team — no SQL knowledge required. QuickSight connects directly to Athena (S3 Parquet data).

QuickSight Feature	Description	ShopFlow Dashboard
SPICE (in-memory engine)	Pre-loads data into QuickSight's fast in-memory store; queries run in seconds even on large datasets	Pre-load 30 days of order data into SPICE; sales team qu
Auto-narratives	AI automatically generates text summaries of charts: "Revenue increased 23% compared to last week, driven by [illegible]	Weekly executive summary auto-generated [illegible]
ML Insights	Built-in anomaly detection and forecasting	Revenue anomaly detection: alert if daily revenue drops >
Embedded analytics	Embed QuickSight dashboards in ShopFlow seller portal (sellers see their own sales data)	Seller dashboard: revenue, top products, customer g
Row-level security	Each seller only sees their own data (enforced in QuickSight layer, not just UI)	Seller A cannot see Seller B's revenue even if they guess

18-17 Kinesis Best Practices and Q&A

Question / Best Practice	Details
How many shards do I need?	Shard count = max(write MB/s, read consumers x read MB/s / 2). ShopFlow: 1,000 events/sec
Hot shard problem (one shard getting all traffic)	If partition key has low cardinality (e.g., only 5 product categories), all events go to 5 shards wh
Kinesis vs SQS — when to use each?	Kinesis: multiple consumers reading the SAME data simultaneously (trending engine AND frau
Lambda Kinesis consumer error handling	By default, a failed batch blocks the shard until the batch succeeds or expires. Set bisectBatch
Firehose buffer size vs latency tradeoff	Firehose buffers for up to 5 minutes OR 128 MB before delivering to S3. Smaller buffer = more

18-18 Kinesis Shard Management — Scaling and Resharding

As ShopFlow grows, the user event volume grows too. Kinesis shards can be split (increase capacity) or merged (reduce cost). On-demand mode handles this automatically.

Scenario	Action	Command / CDK	Consideration
Traffic increases 3x (new marketing campaign)	Split shards: double from 10 to 20	aws kinesis split-shard --stream-name shopflow-user-events	Splitting takes ~30 seconds per shard; 00
Traffic drops after Black Friday (post-season)	Merge shards (cost reduction): reduce from 20 back to 10	aws kinesis merge-shards --stream-name shopflow-user-events	Merging takes ~30 seconds; only adjacent
Unpredictable traffic spikes	Use On-demand mode (auto-scaling)	CDK: streamMode(StreamMode.ON_DEMAND) (no shard count parameter)	On-demand: up to 200 MB/s write, 400 M
Need to estimate shards for provisioned mode	Formula: shards = max(writeRate/1 MB/s, readRate/2 MB/s × consumers)	Example: 5 MB/s write, 2 consumers = max(5, 2×5/2) = 5 shards	Add 30% buffer: 5 × 1.3 = 7 shards minimum for S

18-19 EventBridge Pipes — Simplified Stream Processing

EventBridge Pipes connects event sources (Kinesis, SQS, DynamoDB Streams) to event targets (Lambda, SQS, SNS, EventBridge) without writing consumer code. For simple transformations, this is faster to implement than Lambda.

```
// CDK: EventBridge Pipe — Kinesis order events -> SNS notification // No Lambda code needed for
simple pass-through with filtering CfnPipe orderEventPipe = CfnPipe.Builder.create(this,
"OrderEventPipe") .name("shopflow-order-events-pipe") .roleArn(pipeRole.getRoleArn()) //
Source: Kinesis stream .source(orderEventsStream.getStreamArn())
.sourceParameters(CfnPipe.PipeSourceParametersProperty.builder()
.kinesisStreamParameters(CfnPipe.PipeSourceKinesisStreamParametersProperty.builder()
.startingPosition("LATEST") .batchSize(10) .build()) .build()) // Filter: only process
ORDER_PLACED events (ignore other order events) .enrichment(null)
.filter(CfnPipe.FilterCriteriaProperty.builder()
.filters(List.of(CfnPipe.FilterProperty.builder()
.pattern("{"data":{"action":["ORDER_PLACED"]}}") .build())) .build()) // Optional enrichment:
call Lambda to add product details to event // (omitting for simple case) // Target: SNS topic
for order notifications .target(orderNotificationTopic.getTopicArn())
.targetParameters(CfnPipe.PipeTargetParametersProperty.builder()
.snsTopicParameters(CfnPipe.PipeTargetSnsTopicParametersProperty.builder() .build()) .build())
.build(); // This pipe routes ORDER_PLACED events from Kinesis to SNS // without writing a
Lambda consumer or managing Kinesis checkpoints // Use case: simpler than Lambda when no
transformation is needed
```

18-20 DynamoDB Streams — Change Data Capture

DynamoDB Streams captures every change (insert, update, delete) to a DynamoDB table as a stream of records. ShopFlow uses DynamoDB Streams to propagate cart changes to OpenSearch for real-time inventory updates.

Feature	Description	ShopFlow Use
Change types captured	INSERT (new item), MODIFY (item updated), REMOVE (item deleted)	Cart item added/removed; order status changed from PENDING t
Record contents	OLD_IMAGE (before update), NEW_IMAGE (after update), or KEYS_ONLY	MODIFY with NEW_IMAGE: see the new cart contents after each
Retention	24 hours (fixed; cannot be extended)	Lambda processes within minutes; 24h is enough for retry buffer
Consumer integration	Lambda event source mapping (same as Kinesis)	Lambda reads DynamoDB Streams and updates OpenSearch pro
Kinesis replication	DynamoDB Streams -> Kinesis: use table's kinesis stream config	ShopFlow uses Kinesis replication for order status changes (longer retention, 3 con

```
// CDK: Enable DynamoDB Streams + Lambda trigger Table cartTable = Table.Builder.create(this,
"CartTable") .tableName("shopflow-cart")
.partitionKey(Attribute.builder().name("userId").type(AttributeType.STRING).build())
.stream(StreamViewType.NEW_IMAGE) // capture new state after each change .build(); // Lambda
reads cart changes and updates OpenSearch inventory Function cartSyncLambda =
Function.Builder.create(this, "CartSyncLambda")
.functionName("shopflow-cart-to-opensearch-sync") .runtime(Runtime.JAVA_21)
.handler("com.shopflow.sync.CartSyncHandler::handleRequest") .build();
cartSyncLambda.addEventSource(DynamoEventSource.Builder.create(cartTable)
```

```
.startingPosition(StartingPosition.LATEST) .batchSize(50) .bisectBatchOnFunctionError(true)
.build()); cartTable.grantStreamRead(cartSyncLambda);
```

18-21 ShopFlow Data Lake Architecture

ShopFlow's data lake on S3 stores all historical data — user events, orders, product catalog, system logs — in a queryable format. The Kinesis Firehose pipelines feed data into the lake continuously.

Layer	Description	ShopFlow S3 Structure	Query Tool
Raw (Bronze)	Original events as received; no transformation	s3://shopflow-data-lake/raw/user-events/ (JSON, gzipped)	None (debugging/reprocessing only
Cleaned (Silver)	Validated, deduplicated, PII-masked events in Parquet	s3://shopflow-data-lake/clean/user-events/ (Parquet, partitioned)	Athena (ad-hoc analytics)
Aggregated (Gold)	Pre-aggregated business metrics (daily revenue, weekly MAU)	s3://shopflow-data-lake/gold/daily-revenue/ (Parquet)	Athena + QuickSight dashboards
ML Features	Feature store for recommendation and fraud ML models	s3://shopflow-data-lake/features/user-product-interactions/	SageMaker Feature Store + traini

```
// AWS Glue Data Catalog: register all S3 data lake tables // Glue Crawler runs daily to
discover new partitions and update table schema CfnCrawler glueCrawler =
CfnCrawler.Builder.create(this, "DataLakeCrawler") .name("shopflow-datalake-crawler")
.role(glueRole.getRoleArn()) .databaseName("shopflow_analytics")
.targets(CfnCrawler.TargetsProperty.builder() .s3Targets(List.of(
CfnCrawler.S3TargetProperty.builder() .path("s3://shopflow-data-lake/clean/") .build()))
.build()) .schedule(CfnCrawler.ScheduleProperty.builder() .scheduleExpression("cron(0 2 * * ?
*)") // run at 2 AM UTC daily .build()) .build(); // After crawling, Athena can query new
partitions automatically
```

18-22 Amazon Redshift — Data Warehousing for ShopFlow

Redshift is AWS's managed data warehouse (OLAP) — optimized for complex analytical queries over billions of rows. ShopFlow uses Redshift for complex multi-table joins that would be slow in Athena.

Aspect	Athena	Redshift
Query model	Serverless: query S3 data; pay per TB scanned	Provisioned or serverless warehouse; columnar storage; pay pe
Best for	Ad hoc queries; infrequent analysis; S3 native data	Regular complex queries; joins across many tables; BI tool con
Performance	Good for simple queries; slow for complex multi-table joins over large datasets	Excellent for complex joins; columnar compression; massively p
Cost model	$5/TB scanned; no idle cost	From $0.36/hour (dc2.large) or $0.375/RPU-hour (serverless)
ShopFlow use	Daily ad-hoc analysis, A/B test queries, one-off data pulls	Weekly revenue reporting, complex customer lifetime value que

18-23 Streaming and Analytics Best Practices Summary

Practice	Details
Use high-cardinality partition keys	Partition key = shard assignment. userId or sessionId distributes evenly. category (5 values) would
Design for idempotency in consumers	Kinesis guarantees at-least-once delivery — the same record may be processed twice. Lambda co
Archive raw events before processing	Store a raw copy of all Kinesis events in S3 via Firehose BEFORE processing. If the Lambda consu
Use Parquet for Athena (not JSON)	Parquet columnar format is 10-20x cheaper to query in Athena vs JSON because Athena only read
Kinesis On-demand for unpredictable traffic	On-demand mode auto-scales shards from 4 to 200 based on traffic. More expensive per MB at co
Separate streams by data sensitivity	Do not mix PII-containing events (user actions with email/address) with non-PII events (product vie

18-24 Windowed Aggregations — Revenue per Minute Dashboard

Flink windowed aggregations compute metrics over sliding or tumbling time windows. ShopFlow's live revenue dashboard uses a 1-minute tumbling window to show orders-per-minute and revenue-per-minute in real-time.

```
// Flink: 1-minute tumbling window — orders per minute for live dashboard
DataStream[OrderEvent] orders = env.addSource( new
FlinkKinesisConsumer[OrderEvent]("shopflow-order-events", ...)); // Tumbling window:
non-overlapping 1-minute buckets DataStream[MinuteRevenueSummary] revenuePerMinute = orders
.filter(e -> "ORDER_PLACED".equals(e.action())) .assignTimestampsAndWatermarks(
WatermarkStrategy.[OrderEvent]forBoundedOutOfOrderness(Duration.ofSeconds(5))
.withTimestampAssigner((e, ts) -> e.timestamp().toEpochMilli()))
.windowAll(TumblingEventTimeWindows.of(Time.minutes(1))) .aggregate(new RevenueAggregator(),
new MinuteSummaryWindowFunction()); // At end of each 1-minute window, emit one summary record:
// { windowStart: "14:23:00", windowEnd: "14:24:00", // orderCount: 847, revenueUsd: 42350.25,
avgOrderValue: 50.06 } // Sink: write to DynamoDB for dashboard polling
revenuePerMinute.addSink(new DynamoDBSink[MinuteRevenueSummary]( "shopflow-live-metrics",
summary -> Map.of( "metricKey", AttributeValue.fromS("revenue:" + summary.windowStart()),
"orderCount", AttributeValue.fromN(String.valueOf(summary.orderCount())), "revenueUsd",
AttributeValue.fromN(String.valueOf(summary.revenueUsd())), "ttl",
AttributeValue.fromN(String.valueOf( Instant.now().getEpochSecond() + 3600)) // expire after 1
hour ))); // Live dashboard React component polls DynamoDB every 5 seconds: // GET
/api/live-metrics -> reads last 60 minutes of MinuteRevenueSummary // Renders sparkline chart
of revenue per minute
```

Window Type	Description	ShopFlow Use Case
Tumbling (non-overlapping)	Fixed-size, non-overlapping windows: [0-60s], [60-120s], [120-180s]	Revenue per minute dashboard; hourly order count for dash
Sliding (overlapping)	Windows slide by a step: [0-60s], [10-70s], [20-80s] (overlap)	Trending products: "views in last 5 minutes" updated every
Session	Window closes after N seconds of inactivity for each key	Checkout abandonment: session closes after 10 min idle ->

18-25 Amazon MSK — Managed Kafka as a Kinesis Alternative

Amazon MSK (Managed Streaming for Apache Kafka) is the AWS-managed Apache Kafka service. If your team already uses Kafka or needs Kafka-specific features (consumer groups, topic compaction), MSK is the alternative to Kinesis.

Aspect	Amazon Kinesis	Amazon MSK (Kafka)
API	AWS proprietary (PutRecord, GetRecords)	Apache Kafka standard API (Kafka producer/consumer librarie
Ecosystem	AWS-native (works seamlessly with Lambda, Firehose, CDK)	Kafka ecosystem: Kafka Connect, Kafka Streams, ksqlDB, Co
Ops overhead	Serverless (no servers to manage; shards, not brokers)	Brokers managed by AWS but you choose instance types, sto
Consumer model	Consumer reads from checkpoint (sequence number)	Consumer groups with offsets; flexible multi-consumer pattern
ShopFlow decision	Kinesis chosen: fully serverless, native AWS integration, no Kafka expertise required	MSK would be chosen if migrating existing Kafka workload, n

18-26 Real-Time Personalization — Recommendations from Events

ShopFlow's personalization engine uses a combination of real-time Kinesis events and pre-computed ML recommendations to surface relevant products to each user.

Approach	Latency	ShopFlow Implementation	Freshness
Collaborative filtering (batch ML)	Trained nightly by SageMaker, results loaded into ElastiCache	"Customers who bought X also bought Y" — based on purchase history across all users	Updated daily; misses very recent
Content-based (real-time signal)	< 1 second via Redis	Kinesis trending engine updates Redis sorted set; recommendations read trending scores	Updated every 30 seconds from click
Session-based (short-term)	< 100ms from DynamoDB	Recent session actions (product views in last 10 min) boost relevance in real-time ranking	Updated on every page view

```
// Spring Boot product-svc: blend real-time signals with ML recommendations public
List[ProductId] getPersonalizedRecommendations(String userId, int limit) { // 1. Get ML-based
recommendations (cached in Redis, updated daily) List[ScoredProduct] mlRecs =
redisService.getCollaborativeRecs(userId); // 2. Get trending products from Redis sorted set
(updated every 30s by Lambda) String window = currentFiveMinuteWindow(); List[ScoredProduct]
trending = redisService.getTrending(window, 20); // 3. Get current session context from
DynamoDB (last 10 viewed products) List[String] sessionViews =
dynamoService.getRecentViews(userId, 10); // 4. Blend scores with weights Map[String, Double]
```

```
blendedScores = new HashMap(); mlRecs.forEach(p -> blendedScores.merge(p.id(), p.score() * 0.6,
Double::sum)); // 60% ML trending.forEach(p -> blendedScores.merge(p.id(), p.score() * 0.3,
Double::sum)); // 30% trending // Boost products similar to session views: 10%
addSessionBoost(blendedScores, sessionViews, 0.1); return blendedScores.entrySet().stream()
.sorted(Map.Entry.[String, Double]comparingByValue().reversed()) .limit(limit) .map(e -> new
ProductId(e.getKey())) .collect(Collectors.toList()); }
```

18-27 Black Friday Real-Time Analytics Dashboard

On Black Friday, ShopFlow runs a dedicated real-time operations dashboard fed directly from Kinesis. The CEO and growth team monitor it on a large screen throughout the day.

Metric	Data Source	Update Frequency	Black Friday Target
Orders per minute	Kinesis order-events -> Flink 1-min window -> DynamoDB	Every 1 minute	> 500 orders/minute at peak (vs 30/min normal)
Revenue per minute	Same Flink job -> DynamoDB	Every 1 minute	> $30,000/minute at peak
Checkout conversion rate	Kinesis user-events: CHECKOUT_START vs ORDER_PLACED ratio	Every 5 minutes	> 72% (vs 68% normal; users are more intentional
Cart abandonment rate	Flink session detection -> SNS -> Lambda -> DynamoDB	Every 5 minutes	< 25% (normal: 35%; BF shoppers are more motiva
Top trending products	Lambda trending engine -> Redis ZRANGE -> API	Every 30 seconds	Top 10 rotate rapidly; detect new viral products ear
Inventory at risk	DynamoDB Streams -> Lambda -> inventory check	Every 1 minute	Alert if any trending product has < 50 units remainin
ECS service health	CloudWatch metrics (existing monitoring)	Every 1 minute	All services green; p99 latency < 500ms

18-28 Chapter Summary — ShopFlow Streaming Analytics

You now understand ShopFlow's complete real-time analytics stack. Here is what each layer provides.

Layer	Service	ShopFlow Role
Event ingestion	Kinesis Data Streams (10 shards)	shopflow-user-events and shopflow-order-events receive millions of events/day from S
Real-time processing	Lambda (trending engine, fraud detection)	Lambda reads Kinesis events and updates Redis trending sorted set every 10 second
Complex stream processing	Amazon Managed Flink	Checkout abandonment detection (10-min session windows); live revenue dashboard
Near-real-time archival	Kinesis Firehose	Delivers all events to S3 as Parquet every 5 minutes; enables cheap Athena queries o
Batch analytics	Athena + Glue	SQL queries over S3 Parquet archives; A/B test analysis; cohort analysis; product per
Business intelligence	QuickSight	Self-service dashboards for business team and embedded seller analytics portal
Data warehouse	Redshift	Complex multi-table joins; customer lifetime value; weekly revenue reporting for financ

18-29 Kinesis Client Library (KCL) — Advanced Consumer Patterns

The Kinesis Client Library (KCL) is a Java library that handles all the complexity of reading from Kinesis at scale: lease management, checkpointing, shard splits/merges, and exactly-once processing. ShopFlow uses KCL for the analytics service that builds hourly product reports.

KCL Feature	What It Does	Without KCL
Lease management	Automatically distributes shards across multiple consumer instances;	You must manually track which instance reads which
Checkpointing	Saves last-processed sequence number to DynamoDB; after restart, resumes from checkpoint instead	Manual DynamoDB writes; risk of duplicate or skipped
Shard change detection	Detects shard splits and merges in real-time; automatically starts processing child shards	Manual polling of DescribeStream; easy to miss new s
Multi-language support	KCL 2.x supports Java natively; MultiLangDaemon for Python/Node	Pure Java or manual HTTP calls to Kinesis API

```
// KCL 2.x — ShopFlow hourly product analytics consumer // build.gradle: implementation
"software.amazon.kinesis:amazon-kinesis-client:2.5.8" public class HourlyAnalyticsProcessor
implements ShardRecordProcessor { private String shardId; public void
```

```
initialize(InitializationInput input) { this.shardId = input.shardId(); log.info("Analytics processor initialized for shard: {}", shardId); } public void processRecords(ProcessRecordsInput input) { List[KinesisClientRecord] records = input.records(); // Batch process all records in this poll Map[String, Long] productViewCounts = records.stream() .filter(r -> isProductViewEvent(r)) .collect(Collectors.groupingBy( r -> extractProductId(r), Collectors.counting())); // Write aggregated counts to DynamoDB in one BatchWriteItem call analyticsDao.batchIncrementProductViews(productViewCounts); // IMPORTANT: checkpoint after successful processing // KCL saves this to DynamoDB so we resume here after restart input.checkpointer().checkpoint(); } public void leaseLost(LeaseLostInput input) { // Another instance took over this shard — stop processing immediately log.info("Lease lost for shard: {}", shardId); } public void shardEnded(ShardEndedInput input) { // Shard was split or merged — checkpoint at SHARD_END to release lease input.checkpointer().checkpoint(ExtendedSequenceNumber.SHARD_END); } }
```

18-30 Enhanced Fan-Out — Dedicated Bandwidth per Consumer

By default, all consumers of a Kinesis shard share 2 MB/s read throughput. Enhanced Fan-Out gives each registered consumer its own dedicated 2 MB/s per shard, enabling multiple consumers to read at full speed simultaneously without competing.

Scenario	Standard Consumer	Enhanced Fan-Out Consumer
Bandwidth per shard	2 MB/s SHARED across all consumers (if 3 con[illegible]	2 MB/s DEDICATED per consumer [illegible]
Latency	~200ms average (polling interval)	~70ms average (HTTP/2 push — server pushes records as
Cost	Free (included in shard-hour pricing)	$0.015 per consumer-shard-hour PLUS $0.013 per GB deli
ShopFlow use case	Single consumers with low throughput needs (e.[illegible]	[illegible] consumers running simultaneously (Lamb

tip When to Use Enhanced Fan-Out

Enable Enhanced Fan-Out when: (1) you have 2 or more consumers reading the same stream and each needs full throughput, OR (2) your consumer is falling behind (GetRecords returns empty / iterator age is growing). For ShopFlow, the shopflow-order-events stream has 3 consumers (Lambda, Flink, KCL analytics) — Enhanced Fan-Out prevents them from starving each other.

18-31 Cross-Region Analytics — Disaster Recovery for Data Streams

ShopFlow's analytics data is mission-critical. If us-east-1 has an outage during Black Friday, the team needs analytics data to remain accessible. Cross-region replication provides resilience for both real-time processing and the data lake.

Layer	Replication Approach	RTO / RPO
Kinesis Data Streams	No built-in cross-region replication. Approach: dual-w[illegible]	[illegible]
S3 Data Lake (Firehose o[illegible]	[illegible] S3 Cross-Region Replication (CRR) from sh[illegible]	[illegible]
Redshift	Redshift cross-region snapshot copy: automated hou[illegible]	[illegible]
DynamoDB (trending/me[illegible]	[illegible]moDB Global Tables: active-active replication w[illegible]	[illegible]

18-32 Ops Runbook — Common Kinesis Issues and Fixes

When the on-call alarm fires at 2 AM on Black Friday, you need to resolve issues fast. Here are the three most common Kinesis problems and their step-by-step fixes.

Symptom	Root Cause	Fix
ProvisionedThroughputEx[illegible]	[illegible]	[illegible]
Lambda iterator age growing	[illegible]	[illegible]
Firehose S3 delivery failures	[illegible]	[illegible]

18-33 Review Q&A

Q	A
What is a Kinesis shard and how many does ShopFlow use?	Any doubts please ask. Kinesis capacity: 1 MB/s writes, 2 MB/s reads. ShopFlow uses 10 shards
Why does ShopFlow use Firehose in addition to Lambda for S3 archival?	Firehose provides automatic batching and writing any consumer code. It buffers records, conve
What is the difference between Kinesis and SQS?	Kinesis is a streaming platform designed for ordered, replayable event streams that multiple consum
How does ShopFlow detect trending products in real-time?	A Lambda function reads from the Kinesis user-events stream. For each PRODUCT_VIEW event, it
When would you choose Managed Flink over Lambda for stream processing?	Use Flink when you need (1) stateful windowed aggregations (e.g., revenue per minute, session det

18-34 Monitoring Kinesis — Key Metrics and Alarms

CloudWatch provides built-in Kinesis metrics at no extra cost. ShopFlow monitors these 6 metrics and has alarms on the two most critical ones.

CloudWatch Metric	Namespace	What It Measures	ShopFlow Alarm Threshold
GetRecords.IteratorAgeMilliseconds	AWS/Kinesis	Age of oldest unread record in the stream. If this grows, your consumer is falling b	Alarm if > 60,000ms (1 minute)
WriteProvisionedThroughputExceeded	AWS/Kinesis	Number of records rejected due to exceeding shard write capacity (1 MB/s or 1,00	Alarm if sum > 0 for 1 minute
PutRecords.ThrottledRecords	AWS/Kinesis	Count of records throttled within a PutRecords batch	Monitor; track trend; refactor producer
IncomingRecords	AWS/Kinesis	Total records written per second across all shards	Monitor to predict when to add shards (a
ReadProvisionedThroughputExceeded	AWS/Kinesis	Consumer reads throttled (exceeded 2 MB/s per shard or 5 GetRecords per sha	Alarm if > 0 for 5 minutes (consider enhanc
MillisBehindLatest (Lambda)	AWS/Lambda	Lambda-specific: how far behind the latest record the Lambda is. Equivalent to Iterator	Alarm if > 30,000ms for 5 min (Lambd

■ Iterator Age

The IteratorAgeMilliseconds metric tells you how "stale" your consumer is. It is the age of the oldest record your consumer has not yet processed. For ShopFlow's fraud detection Lambda, we want iterator age close to zero — meaning the Lambda is processing events nearly as fast as they arrive. An iterator age of 60,000ms means the Lambda is 1 minute behind, which could let fraudulent orders through undetected.

Example: Black Friday scenario: at 10,000 orders/minute, if Lambda processing takes 2 seconds per batch, iterator age grows. Solution: increase Lambda parallelization factor to 5 (Lambda creates 5 concurrent executions per shard).

18-35 End-to-End Event Flow — From Click to Dashboard

Follow a single product view event through ShopFlow's entire analytics pipeline — from the customer's browser click to the CEO's live dashboard, with latency at each hop.

Step	Component	Latency	What Happens
1. User action	React browser -> Spring Boot product-svc	< 50ms	Customer views product page. product-svc handles the
2. Kinesis ingestion	Kinesis Data Streams (shopflow-user-events)	< 5ms	Record written to shard (determined by productId partiti
3. Lambda trigger	Lambda trending engine	< 200ms (Lambda cold-start included)	Lambda reads batch of events from Kinesis. For each P
4. API serving	Spring Boot product-svc GET /api/trending	< 10ms	API reads Redis sorted set via ZREVRANGE trending:2
5. Firehose archival	Kinesis Firehose -> S3	5 minutes buffer	Firehose batches records for 5 minutes, converts to Pa
6. Dashboard query	Athena over S3 Parquet	5-30 seconds	Business analyst runs: SELECT product_id, COUNT(*)

18-36 Hands-On Lab — Build the Complete Trending Engine

In this lab you will create a Kinesis stream, write a Spring Boot producer, deploy a Lambda consumer that updates Redis, and verify trending products appear in the API. Estimated time: 45 minutes.

■ Lab Steps: Trending Engine

1. ('Step 1 — Create Kinesis stream', 'aws kinesis create-stream --stream-name shopflow-user-events-lab --shard-count 2\naws kinesis wait stream-exists --stream-name shopflow-user-events-lab')

2. ('Step 2 — Create Redis ElastiCache cluster (if not already running)', 'aws elasticache create-cache-cluster \\\n --cache-cluster-id shopflow-trending-lab \\\n --engine redis \\\n --cache-node-type cache.t3.micro \\\n --num-cache-nodes 1')

3. ('Step 3 — Deploy Lambda trending engine (ZIP deployment)', 'cd lambda/trending-engine\nzip -r function.zip .\naws lambda create-function \\\n --function-name shopflow-trending-engine-lab \\\n --runtime java21 \\\n --handler com.shopflow.TrendingHandler::handleRequest \\\n --zip-file fileb://function.zip \\\n --role arn:aws:iam::ACCOUNT:role/shopflow-lambda-role')

4. ('Step 4 — Add Kinesis trigger to Lambda', 'aws lambda create-event-source-mapping \\\n --function-name shopflow-trending-engine-lab \\\n --event-source-arn arn:aws:kinesis:us-east-1:ACCOUNT:stream/shopflow-user-events-lab \\\n --starting-position LATEST \\\n --batch-size 100 \\\n --parallelization-factor 2')

5. ('Step 5 — Send test events and verify', '# Send 50 product view events for product-123\nfor i in $(seq 1 50); do\n aws kinesis put-record \\\n --stream-name shopflow-user-events-lab \\\n --partition-key "product-123" \\\n --data $(echo \'{"action":"PRODUCT_VIEW","productId":"product-123","userId":"user-1"}\' | base64)\ndone\n\n# After ~10 seconds, check Redis sorted set:\nredis-cli ZREVRANGE trending:$(date +%Y%m%d:%H%M -d "1 minute ago") 0 9 WITHSCORES')

6. ('Step 6 — Verify CloudWatch metrics', '# Check Lambda invocations\naws cloudwatch get-metric-statistics \\\n --namespace AWS/Lambda \\\n --metric-name Invocations \\\n --dimensions Name=FunctionName,Value=shopflow-trending-engine-lab \\\n --start-time $(date -u -d "10 minutes ago" +%Y-%m-%dT%H:%M:%S) \\\n --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \\\n --period 60 \\\n --statistics Sum\n\n# Expected output: sum > 0 (Lambda was triggered by Kinesis events)')

18-37 Cost Optimization — Right-Sizing the Analytics Stack

ShopFlow's analytics stack costs roughly $2,400/month at steady state. Here is where the money goes and how to reduce it without sacrificing functionality.

Service	Monthly Cost Estimate	Largest Cost Driver	Optimization
Kinesis Data Streams (15 shards)	~$320/month	Shard-hours (each shard: $0.015/hour x 720h)	Use on-demand mode during development; switch to
Kinesis Firehose	~$45/month	Data ingested ($0.029/GB; ~1.5 TB/month)	Enable Snappy compression (reduces data size ~60
Athena	~$130/month	Data scanned per query ($5/TB scanned)	Partition S3 data by date (WHERE date='2024-01-12
Amazon MSK / Managed Flink	~$380/month	Flink Application Studio KPU-hours ($0.11/KPU-hour)	Right-size KPUs: start at 4 KPUs; scale up only durin
QuickSight	~$168/month	Author seats ($18/author/month x 5 authors) + reading sessions ($0.30/session)	Use reader pricing ($0.30/session) for business user
Redshift Serverless	~$290/month	RPUs consumed (base RPU x hours)	Set max RPU cap. Schedule pause/resume (pause r

info Total Monthly Analytics Cost

Estimated steady-state: ~$2,400/month. During Black Friday week: ~$4,100/month (extra Kinesis shards, higher Flink KPUs, more Athena queries). On a $1M ARR product, this is 2.9% of revenue — well within typical cloud cost budgets of 10-20% of ARR for infrastructure.

18-38 Production Readiness Checklist — Kinesis and Analytics

Before you go live with ShopFlow's streaming analytics stack, verify every item on this checklist. Each row is a potential production incident if skipped.

Area	Checklist Item	Why It Matters
Kinesis Streams	Shard count sized to 50% of peak throughput (never >80%)	At 80%+ utilization, any traffic spike causes ProvisionedThrou

Area	Checklist Item	Why It Matters
Kinesis Streams	Producer has exponential backoff + jitter for PutRecords	Without backoff, throttled producers retry immediately, creatin
Kinesis Streams	Data retention set to 7 days (default is 24h)	24h retention means a consumer outage lasting > 24h loses d
Lambda Consumer	Parallelization factor set to 3-5 for high-throughput streams	Without parallelization, a single Lambda processes all records
Lambda Consumer	Dead-letter queue (SQS DLQ) configured for Lambda	Failed batches (parse errors, downstream timeouts) are retrie
Firehose	S3 error prefix configured: prefix=firehose-errors/	Without error prefix, delivery failures are invisible. You only dis
Firehose	Parquet conversion enabled with Glue schema	JSON archival costs 10x more in Athena queries. Parquet red
Athena	S3 data partitioned by date (year/month/day/hour)	Unpartitioned data forces full table scans on every query. A 1-
Redshift	Automated snapshots with cross-region copy enabled	Single-region snapshots only protect against data corruption,
Monitoring	CloudWatch alarms on IteratorAgeMilliseconds and WriteProvisionedThroughputExceeded	These are the two early warning signs of streaming problems.